

EXPLORING STATISTICS MODULE

This module provides functions for calculating mathematical statistics of numeric (Real-valued) data.

Mean Calculation

```
In [2]: import statistics

# initializing list
li = [1, 2, 3, 3, 2, 2, 1]

# using mean() to calculate average of list
# elements
print ("The average of list values is : ",end="")
print (statistics.mean(li))
print (statistics.harmonic_mean(li))
```

```
The average of list values is : 2
1.7142857142857142
```

Harmonic mean = The reciprocal of the arithmetic mean() of the reciprocals of the data.

The harmonic mean is calculated as follows:

If you have four values (a, b, c and d) - it will be equivalent to $4 / (1/a + 1/b + 1/c + 1/d)$.

Median

```
In [3]: data1 = (2, 3, 4, 5, 7, 9, 11)
print("Median of data-set 1 is",statistics.median(data1))
data2 = (-1, -2, -3, -4, 4, 3, 2, 1)
print("Median of data-set 2 is",statistics.median(data2))
```

```
Median of data-set 1 is 5
Median of data-set 2 is 0.0
```

Median Low

The median_low() function returns the median of data in case of odd number of elements, but in case of even number of elements, returns the lower of two middle elements. If the passed argument is empty, StatisticsError is raised

Example: Python code to calculate Median Low

```
In [4]: import statistics

# simple list of a set of integers
set1 = [1, 3, 3, 4, 5, 7]

# Print median of the data-set

# Median value may or may not
# lie within the data-set
print("Median of the set is % s"
      % (statistics.median(set1)))

# Print Low median of the data-set
print("Low Median of the set is % s "
      % (statistics.median_low(set1)))
```

Median of the set is 3.5
Low Median of the set is 3

Median High

The median_high() function returns the median of data in case of odd number of elements, but in case of even number of elements, returns the higher of two middle elements. If passed argument is empty, StatisticsError is raised.

Example: Python code to calculate Median High

```
In [5]: import statistics

# simple list of a set of integers
set1 = [1, 3, 3, 4, 5, 7]

# Print median of the data-set

# Median value may or may not
# lie within the data-set
print("Median of the set is %s"
      % (statistics.median(set1)))

# Print high median of the data-set
print("High Median of the set is %s "
      % (statistics.median_high(set1)))
```

Median of the set is 3.5
High Median of the set is 4

Mode

It is the value that has the highest frequency in the given data set.

```
In [6]: # tuple of positive integer numbers
data1 = (2, 3, 3, 4, 5, 5, 5, 5, 6, 6, 6, 7)

# tuple of a set of floating point values
data2 = (2.4, 1.3, 1.3, 1.3, 2.4, 4.6)
print("Mode of data set 1 is % s" % (st.mode(data1)))
print("Mode of data set 2 is % s" % (st.mode(data2)))
```

```
Mode of data set 1 is 5
Mode of data set 2 is 1.3
```

statistics.multimode(data)

Return a list of the most frequently occurring values in the order they were first encountered in the data. Will return more than one result if there are multiple modes or an empty list if the data is empty:

```
In [7]: st.multimode('aabbbbccdddeeffffgg')
```

```
Out[7]: ['b', 'd', 'f']
```

statistics.pvariance(data, mu=None)

Return the population variance of data

```
In [8]: data = [0.0, 0.25, 0.25, 1.25, 1.5, 1.75, 2.75, 3.25]
st.pvariance(data)
```

```
Out[8]: 1.25
```

statistics.quantiles(data, *, n=4, method='exclusive')

Divide data into n continuous intervals with equal probability. Returns a list of n - 1 cut points separating the intervals.

Set n to 4 for quartiles (the default). Set n to 10 for deciles. Set n to 100 for percentiles which gives the 99 cuts points that separate data into 100 equal sized groups. Raises StatisticsError if n is not least 1.

```
In [9]: data = [105, 129, 87, 86, 111, 111, 89, 81, 108, 92, 110,
               100, 75, 105, 103, 109, 76, 119, 99, 91, 103, 129,
               106, 101, 84, 111, 74, 87, 86, 103, 103, 106, 86,
               111, 75, 87, 102, 121, 111, 88, 89, 101, 106, 95,
               103, 107, 101, 81, 109, 104]
[round(q, 1) for q in st.quantiles(data, n=10)]
```

```
Out[9]: [81.0, 86.2, 89.0, 99.4, 102.5, 103.6, 106.0, 109.8, 111.0]
```

Measure of Variability

Till now, we have studied the measure of central tendency but this alone is not sufficient to describe the data. To overcome this we need the measure of variability. The measure of variability is known as the spread of data or how well our data is distributed. The most common variability measures are:

- Range
- Variance

- Standard deviation

```
In [10]: arr = [1, 2, 3, 4, 5]

#Finding Max
Maximum = max(arr)
# Finding Min
Minimum = min(arr)

# Difference Of Max and Min
Range = Maximum-Minimum
print("Maximum = {}, Minimum = {} and Range = {}".format(
    Maximum, Minimum, Range))
```

Maximum = 5, Minimum = 1 and Range = 4

Variance

It is defined as an average squared deviation from the mean. It is calculated by finding the difference between every data point and the average which is also known as the mean, squaring them, adding all of them, and then dividing by the number of data points present in our data set.

```
In [11]: sample1 = (1, 2, 5, 4, 8, 9, 12)
print("Variance of Sample1 is ", st.variance(sample1))
```

Variance of Sample1 is 15.80952380952381

Standard Deviation

It is defined as the square root of the variance.

```
In [12]: sample1 = (1, 2, 5, 4, 8, 9, 12)
print("The Standard Deviation of Sample1 is", (st.stdev(sample1)))
```

The Standard Deviation of Sample1 is 3.9761191895520196

statistics.covariance(x, y, /)

Return the sample covariance of two inputs x and y. Covariance is a measure of the joint variability of two inputs.

Both inputs must be of the same length (no less than two), otherwise StatisticsError is raised.

Examples:

```
In [13]: x = [1, 2, 3, 4, 5, 6, 7, 8, 9]
y = [1, 2, 3, 1, 2, 3, 1, 2, 3]
statistics.covariance(x, y)
z = [9, 8, 7, 6, 5, 4, 3, 2, 1]
statistics.covariance(x, z)
```

Out[13]: -7.5

```
In [14]: x = [1, 2, 3, 4, 5, 6, 7, 8, 9]
         y = [9, 8, 7, 6, 5, 4, 3, 2, 1]
         st.correlation(x, x)
```

```
Out[14]: 1.0
```

```
In [ ]:
```